

Version Control System (Mini-Git)

Patterns: Command Pattern, Composite Pattern, Strategy Pattern (Diff/Hash), Immutable Object | Domain: Infrastructure / Immutable DAG

Core Requirements & Features	Core Domain Classes & Interfaces
• Content Addressable Storage: Store files as SHA-1/SHA-256 hashed immutable Blob objects. • Tree & Commit Structure: Directory trees mapping files/subtrees; Commits forming a DAG. • Branch & HEAD Pointers: Create branches, update HEAD, checkout commits, and stage changes.	• GitObject: Abstract base class for Blob, Tree, & Commit. • Blob: Immutable byte array wrapper with SHA hash ID. • Tree: Directory listing mapping filenames to GitObject hashes. • Commit: Pointer to root Tree, parent Commit(s), author, & timestamp.

Critical Execution Workflow & Method Trace

1. User modifies file main.py and executes stage('main.py').
 2. VCS calculates SHA-1 hash of main.py content, creates Blob(hash, bytes), and writes to object database.
 3. Adds (filename, blobHash) entry to Staging Area Index map.
 4. User executes commit('Initial commit').
 5. VCS converts Staging Area Index map into Tree object hierarchy and writes Trees to object store.
 6. Creates Commit object pointing to root Tree hash, sets parent = HEAD commit hash.
 7. Updates current Branch reference file (e.g., refs/heads/main) to point to new Commit hash, then advances HEAD.

Concurrency & Thread Safety Mechanics	Design Trade-offs & Interview FAQs
• Object Immutability: GitObjects are strictly read-only once written; zero write contention on existing objects. • Branch Locking: Atomic CAS / file-lock on branch ref updates (refs/heads/main.lock). • Index Thread-Safety: ReentrantReadWriteLock on staging area index file.	• Full Copy vs Delta: Loose objects (full copy) vs Packfiles (delta compression); loose objects speed up writes. • DAG Merging: Simple fast-forward merge vs 3-way merge requiring common ancestor lookup (LCA). • Hash Collisions: SHA-1 collision probability is negligible; SHA-256 used for modern security.